APE 1

I PORTADA

Tema:	Guía APE 1: Diagnóstico de Zonas de Riesgo Urbano
mediante Aprendizaje Basado en Reglas, Lógica Difusa y Algoritmos Genéticos	
Carrera:	Ingeniería de Software
Unidad de Organización Curricular:	PROFESIONAL
Nivel y Paralelo:	Séptimo semestre A
Alumnos participantes:	Velastegui Paul Manjarres David
Asignatura:	Inteligencia Artificial
Docente:	Ing. Nogales Ruben, Mg.

II INFORME DE GUÍA PRÁCTICA

2.1. Objetivos

General:
Desarrollar e implementar un sistema de diagnóstico de zonas de riesgo urbano en la ciudad de Ambato, aplicando de forma integrada aprendizaje basado en reglas, lógica difusa y algoritmos genéticos sobre datos históricos de incidentes, con el fin de clasificar automáticamente el nivel de riesgo por zona y apoyar la toma de decisiones en materia de seguridad ciudadana.

Específicos:

- Definir formalmente las variables de entrada del sistema (robos, microtráfico, vandalismo, accidentes y llamadas al 911), sus rangos válidos y sus categorías lingüísticas, respaldadas en datos históricos por macro-zona, para construir un clasificador difuso que estime el nivel de riesgo urbano en bajo, medio y alto.
- Diseñar e implementar un algoritmo genético que optimice los pesos de ponderación de las variables de entrada, evaluando cada individuo mediante precisión de clasificación sobre un conjunto sintético de prueba y aplicando selección por truncamiento, cruce por promedio, mutación uniforme y elitismo efectivo de un individuo.
- Analizar y documentar los resultados obtenidos por el sistema integrado, identificando las zonas de mayor riesgo en la ciudad, validando la coherencia de los diagnósticos mediante comparación con la distribución histórica de incidentes, y formulando conclusiones y recomendaciones orientadas a la intervención en seguridad ciudadana.

2.2. Modalidad

Presencial.

2.3. Tiempo de duración

Presenciales: 8
No presenciales: 0

2.4. Instrucciones

Realizar una simulación en la que se aplique aprendizaje basado en reglas, lógica difusa y algoritmos genéticos, orientada al diagnóstico de zonas de riesgo en seguridad ciudadana en la ciudad de Ambato.

2.5. Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

- Computador personal con sistema operativo Windows/Linux/macOS
- Python 3.8 o superior
- Librerías principales utilizadas en el código: OSMnx, NetworkX, NumPy, Pandas, SciPy, Folium y Matplotlib
- Soporte `tkinter/Tk` para el modo de diagnóstico detallado con interfaz gráfica
- Editor de código (Visual Studio Code)
- Internet
- Datos históricos del sistema (archivo CSV por macro-zonas)
- Herramientas para generación de documentos en formato PDF

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:
☐ Plataformas educativas
☐ Simuladores y laboratorios virtuales
☐ Aplicaciones educativas
☐ Recursos audiovisuales
☐ Gamificación
■ Inteligencia Artificial
Otros (Especifique): _____

2.6. Actividades por desarrollar

El presente trabajo integra técnicas de Inteligencia Artificial para el análisis de seguridad ciudadana en Ambato. El sistema parte de datos históricos de incidentes por macro-zona (Norte, Sur, Este, Oeste), genera escenarios sintéticos mediante simulación Monte Carlo, aplica un clasificador difuso para estimar el nivel de riesgo urbano, utiliza un algoritmo genético para optimizar pesos y produce reglas explicativas con Apriori/PRISM. A partir de esta arquitectura, las actividades desarrolladas son las siguientes:

1. **Preparación del entorno y estructura del sistema.** Configurar el proyecto en Python e identificar los módulos principales del sistema:
   - Archivo principal `main.py`
   - Configuración en `config.py`
   - Módulos de algoritmos en `src/algoritmos`
   - Módulos de diagnóstico y visualización en `src/core`
   - Utilidades de datos y reportes en `src/utils`
2. **Carga y análisis de datos históricos.** Ejecutar la lectura del archivo de históricos y validar la información por macro-zona para calcular estadísticas base de simulación.
3. **Aplicación de algoritmos de IA.** Integrar en el flujo del diagnóstico los siguientes enfoques:
   - Simulación Monte Carlo para escenarios sintéticos por macro-zona.
   - Algoritmo genético para optimización de pesos de las cinco variables.
   - Clasificación difusa basada en funciones de membresía triangulares y sigmoides.
   - Reglas explicativas mediante Apriori y PRISM.
   - Ajuste espacial mediante análisis de conectividad BFS.

   Producir mapas interactivos, reportes HTML y gráficas persistentes para analizar zonas críticas, comparar escenarios y documentar hallazgos.

3 INFORME TÉCNICO DEL PROYECTO

3.1. Introducción

La seguridad ciudadana constituye uno de los pilares fundamentales del bienestar social en cualquier entorno urbano. En ciudades intermedias de Ecuador como Ambato, la creciente complejidad de los fenómenos delictivos y la heterogeneidad espacial de los incidentes hacen que los enfoques tradicionales de diagnóstico, basados en reportes manuales o estadísticas agregadas sin desagregación geográfica, resulten insuficientes para orientar la toma de decisiones ejecutiva con la precisión y la oportunidad requeridas.

Ante esta problemática, la Inteligencia Artificial (IA) ofrece un conjunto de herramientas metodológicas capaces de procesar volumenes de información heterogénea, capturar la imprecisión inherente a los conceptos de riesgo y generar diagnósticos reproducibles y auditables. En particular, tres paradigmas complementarios resultan especialmente idóneos para este dominio:

1. **Aprendizaje basado en reglas:** técnicas como Apriori y PRISM permiten extraer o formular relaciones del tipo "SI condición ENTONCES riesgo" de manera interpretable. En este proyecto se emplean como capa explicativa posterior al diagnóstico para producir reglas legibles sobre categorías discretizadas de las variables [4, 5].

2. **Lógica difusa:** propuesta originalmente por Zadeh [1] en 1965, la lógica difusa gestiona de forma natural la imprecisión lingüística de conceptos como "zona de alto riesgo" o "nivel de microtráfico moderado". A través de funciones de membresía, transforma valores cuantitativos en categorías lingüísticas graduales, evitando la artificiosidad de los umbrales booleanos.

3. **Algoritmos genéticos:** inspirados en los mecanismos de la evolución biológica y formalizados por Holland [2], permiten optimizar parámetros en espacios de búsqueda complejos y no diferenciables, como la ponderación de variables de riesgo, sin requerir supuestos restrictivos sobre la forma de la función objetivo.

Este proyecto integra dichos paradigmas en un flujo de procesamiento unificado, construido sobre datos geográficos reales de Ambato obtenidos desde OpenStreetMap y sobre un registro histórico de incidentes delictivos por macro-zona. El resultado es un sistema de diagnóstico capaz de clasificar automáticamente el nivel de riesgo de cada zona de la ciudad, generar visualizaciones georreferenciadas interpretables y servir de insumo para la planificación de intervenciones en seguridad ciudadana.

3.2. Dataset y Fuente de Información

3.2.1. Fuente de los Datos

Los datos empleados en la simulación provienen de dos fuentes complementarias:

- **Datos históricos de incidentes (CSV):** Archivo tabular de trabajo incluido en el repositorio, con registro mensual de cinco categorías de incidentes para cuatro macro-zonas de la ciudad de Ambato (Norte, Sur, Este y Oeste) a lo largo de 12 meses. Este archivo funciona como base histórica interna del prototipo y no se encuentra enlazado en tiempo real con una fuente institucional externa.

- **Grafo vial de OpenStreetMap (OSM):** Información geográfica real de la red vial de Ambato, descargada mediante la librería OSMnx [7]. El grafo resultante aporta la base cartográfica sobre la que se superponen los diagnósticos de riesgo y, dependiendo de la fecha de descarga y del recorte espacial, puede contener miles de intersecciones (nodos) y segmentos de calle (aristas).

3.2.2. Variables Consideradas y Justificación

El sistema emplea cinco variables de entrada correspondientes a categorías de incidentes con relevancia directa para la seguridad ciudadana en contextos urbanos como Ambato. La justificación de su inclusión se fundamenta en la literatura especializada en criminología urbana y análisis de seguridad, así como en la disponibilidad de datos históricos desagregados por zona.

Variable	Unidad	Justificación
Robos	casos/mes	Indicador directo de delincuencia patrimonial y variable de referencia básica dentro del diagnóstico implementado.
Microtráfico	casos/mes	Variable asociada a presión delictiva localizada y empleada como señal complementaria dentro del modelo.
Vandalismo	casos/mes	Indicador de deterioro del entorno urbano y de conflictividad visible en el espacio público.
Accidentes viales	casos/mes	Variable de contexto incluida en el prototipo para capturar eventos críticos que también afectan la percepción de riesgo urbano.
Llamadas al 911	llamadas/mes	Indicador agregado de demanda de atención y reporte ciudadano ante eventos problemáticos.

Cuadro 1: Variables de entrada del sistema de diagnóstico

Fuente: Elaboración propia con base en las variables implementadas en el sistema.

La elección de estas cinco variables responde a la disponibilidad de datos históricos sistematizados, a su relevancia como indicadores de riesgo en la literatura de seguridad urbana, y a su capacidad de ser fuzzificadas dentro de universos de discurso bien definidos. Variables como la densidad poblacional o el nivel socioeconómico, aunque relevantes, se excluyeron por la dificultad de obtener datos desagregados con la granularidad espacial requerida.

3.2.3. Estructura del Dataset Histórico

El archivo CSV de históricos contiene 48 filas: 12 meses para cada una de las 4 macro-zonas, con las 5 variables almacenadas como columnas en cada registro. Si se contabilizan los valores numéricos individuales, el conjunto representa $12 \times 4 \times 5 = 240$ observaciones escalares. Para cada macro-zona se calculan cuatro estadísticas descriptivas por variable: media μ, desviación estándar σ, mínimo y máximo. Estos parámetros alimentan la simulación Monte Carlo; las funciones de membresía del clasificador difuso, en cambio, se encuentran definidas directamente en el código.

3.3. Variables y Parámetros del Sistema Difuso

3.3.1. Categorías Lingüísticas y Funciones de Membresía

La clasificación difusa implementada en `src/algoritmos/difuso.py` no utiliza `scikit-fuzzy` ni una base de reglas Mamdani clásica con antecedentes y consecuentes explícitos [6]. La implementación real calcula directamente tres grados de pertenencia globales (bajo, medio y alto) a partir de las cinco variables de incidentes y de los pesos generados por el algoritmo genético.

Antes de evaluar la membresía, cada variable se normaliza respecto a un máximo fijo de dominio:

$$x'_v = \frac{x_v}{M_v} \tag{1}$$

donde M_v es el máximo permitido para la variable v en el código. La Tabla 2 resume los dominios y el tipo de función asociado a cada variable.

Variable	**Máximo M_v**	**Función principal**
Robos	100	Triangular
Microtráfico	50	Sigmoidal
Vandalismo	80	Triangular
Accidentes	60	Triangular
Llamadas al 911	100	Sigmoidal

Cuadro 2: Dominios máximos y tipo de membresía implementados en el código

Para las variables triangulares, el código usa la función:

$$\mu_{tri}(x;\, a, b, c) = \begin{cases} 0 & \text{si } x \leq a \text{ o } x \geq c \\ \dfrac{x-a}{b-a} & \text{si } a < x \leq b \\ \dfrac{c-x}{c-b} & \text{si } b < x < c \end{cases} \tag{2}$$

Para las variables sigmoides, el código usa:

$$\mu_\sigma(x;\, a, s) = \frac{1}{1 + e^{-s(x-a)}} \tag{3}$$

Con estas familias, el código fija internamente los conjuntos *bajo*, *medio* y *alto* sobre el dominio normalizado $[0, 1]$. En las variables triangulares se usan los triples $(0, 0, 0.4)$, $(0.25, 0.5, 0.75)$ y $(0.6, 1, 1)$; en las variables sigmoides se combinan puntos de inflexión en 0.25, 0.30 y 0.65 con pendiente $s = 12$.

3.3.2. Agregación Difusa y Variable de Salida

La salida intermedia del clasificador se obtiene agregando de forma ponderada los grados de pertenencia de cada variable:

$$\mu_{bajo} = \sum_{v=1}^{5} w_v \cdot \mu_{bajo}^{(v)}(x'_v), \qquad \mu_{medio} = \sum_{v=1}^{5} w_v \cdot \mu_{medio}^{(v)}(x'_v), \qquad \mu_{alto} = \sum_{v=1}^{5} w_v \cdot \mu_{alto}^{(v)}(x'_v) \tag{4}$$

El puntaje numérico ya no se calcula únicamente mediante una combinación ponderada de centros fijos. La versión actual del código define funciones de membresía del consecuente para *riesgo bajo*, *medio* y *alto* sobre el dominio de salida $[0, 100]$, las trunca con los grados μ_{bajo}, μ_{medio} y μ_{alto}, las agrega con el operador máximo y calcula el centroide discreto del área resultante:

$$\mu_{agg}(x) = \text{máx}\left(\text{mín}(\mu_{bajo}, \mu_{cons,bajo}(x)), \text{mín}(\mu_{medio}, \mu_{cons,medio}(x)), \text{mín}(\mu_{alto}, \mu_{cons,alto}(x))\right) \tag{5}$$

$$score = \frac{\sum_{x \in U} x\, \mu_{agg}(x)}{\sum_{x \in U} \mu_{agg}(x)}, \qquad U = \{0, 1, \ldots, 100\} \tag{6}$$

Solo si el área agregada es nula, el código usa como respaldo la defuzzificación por centros representativos fijos $(16.5, 50, 83.5)$.

La etiqueta devuelta por el clasificador difuso se asigna al mayor de los tres grados agregados. En la implementación actual, el parámetro `metricas` existe en la firma de la función, pero no interviene en el cálculo del `score`; el diagnóstico depende únicamente de las cinco variables de incidentes y de los pesos optimizados. Cuando posteriormente se aplica el ajuste por conectividad, el nivel final ya no se toma del máximo grado difuso, sino de la discretización del `score` ajustado con los umbrales 33 y 66.

3.3.3. Reglas Explicativas Apriori y PRISM

El componente basado en reglas se implementa en `src/algoritmos/reglas.py` y se utiliza con fines explicativos posteriores al cálculo del nivel de riesgo, no como motor principal del `score`. El proceso real consta de tres etapas.

Etapa 1 — Discretización Cada variable se discretiza en *bajo*, *medio* o *alto*. Los umbrales usados por el código son:

Variables	Umbrales implementados
Robos, vandalismo, llamadas al 911	bajo si $x < 33$; medio si $33 \leq x < 66$; alto en caso contrario
Microtráfico, accidentes	bajo si $x < 20$; medio si $20 \leq x < 40$; alto en caso contrario

Cuadro 3: Umbrales de discretización del módulo de reglas

Etapa 2 — Generación de dataset transaccional El módulo genera internamente 100 zonas sintéticas. Cada transacción contiene ítems del tipo `robos=alto` y un ítem adicional `riesgo=Nivel`. El nivel se etiqueta con el promedio ponderado fijo:

$$score_{reglas} = 0.35 \cdot robos + 0.15 \cdot microtrafico + 0.25 \cdot vandalismo + 0.10 \cdot accidentes + 0.15 \cdot llamadas911 \tag{7}$$

Etapa 3 — Apriori o PRISM Sobre ese dataset sintético, Apriori calcula itemsets frecuentes y conserva reglas con soporte mínimo 0.10 y confianza mínima 0.70, restringiendo el consecuente a `riesgo=...`. PRISM busca reglas de la forma `SI variable=valor ENTONCES riesgo=clase` con confianza mínima 0.50.

En el pipeline de diagnóstico, la función `obtener_regla_explicativa` devuelve:

- la primera regla Apriori si el nivel final es **Alto**;
- la primera regla PRISM si el nivel final es **Medio**;

- el texto "Sin regla relevante para nivel de riesgo bajo" si el nivel final es **Bajo**.

Por tanto, las reglas se usan como explicación textual del resultado y no como insumo directo del cálculo difuso o del ajuste por conectividad.

3.4. Algoritmo Genético

3.4.1. Motivación

En la implementación actual, el algoritmo genético no optimiza la forma de las funciones de membresía sino únicamente los pesos relativos asignados a las cinco variables de incidentes. Esto convierte el problema en una búsqueda sobre un espacio continuo de cinco dimensiones, donde cada solución candidata representa una combinación convexa de pesos.

- **Espacio continuo acotado:** cada gen debe permanecer en $[0, 1]$.
- **Normalización obligatoria:** los pesos se renormalizan para que $\sum_i w_i = 1$.
- **Función objetivo no lineal:** la calidad de un individuo depende de la precisión de clasificación obtenida sobre un conjunto sintético de prueba.

Los algoritmos genéticos [2] son adecuados para este contexto porque permiten explorar automáticamente múltiples combinaciones de pesos sin requerir gradientes ni una forma analítica simple de la función objetivo.

3.4.2. Codificación del Cromosoma

Cada **individuo** (solución candidata) en el algoritmo genético es un **cromosoma**: un vector de números reales que codifica los pesos de ponderación de las cinco variables de entrada del sistema difuso.

- **Longitud del cromosoma:** 5 genes, uno por variable (w_{robos}, $w_{microtr}$, w_{vandal}, w_{accid}, w_{911}).
- **Dominio de cada gen:** $w_i \in [0, 1]$.
- **Restricción:** $\sum_{i=1}^{5} w_i = 1$ (los pesos se normalizan para formar una combinación convexa).
- **Interpretación:** el cromosoma determina la importancia relativa de cada variable al calcular el índice de riesgo ponderado que alimenta el motor difuso.

Gen	1	2	3	4	5
Variable	Robos	Microtráfico	Vandalismo	Accidentes	Llamadas 911
Dominio	$[0, 1]$	$[0, 1]$	$[0, 1]$	$[0, 1]$	$[0, 1]$

Cuadro 4: Estructura del cromosoma del algoritmo genético

Fuente: Elaboración propia.

3.4.3. Población Inicial

La población inicial de **30 individuos** se genera de manera totalmente aleatoria. Para cada individuo, el código crea cinco valores aleatorios positivos y luego los normaliza para que su suma sea exactamente 1.

- No existe un cromosoma base fijo ni una población parcialmente perturbada.
- La diversidad inicial proviene exclusivamente de la aleatoriedad de los cinco pesos antes de la normalización.

3.4.4. Función de Fitness

La función de fitness implementada en `src/algoritmos/genetico.py` mide únicamente **precisión de clasificación** sobre un dataset sintético interno de 100 ejemplos. Primero se genera un conjunto de prueba con cinco variables y una etiqueta `nivel_real` obtenida mediante:

$$score_{base} = 0.35 \cdot robos + 0.15 \cdot microtrafico + 0.25 \cdot vandalismo + 0.10 \cdot accidentes + 0.15 \cdot llamadas911 \quad (8)$$

y se discretiza el resultado en *Bajo*, *Medio* o *Alto* usando umbrales 33 y 66. Después, cada individuo produce una predicción ponderada y su fitness queda definido como:

$$\text{fitness}(\mathbf{w}) = \frac{\text{número de predicciones correctas}}{\text{número total de ejemplos}} \quad (9)$$

Por tanto, la versión actual del código no utiliza términos explícitos de correlación, coherencia espacial, penalización adicional ni combinación multiobjetivo.

3.4.5. Selección de Padres

La selección de padres no se implementa por torneo. El procedimiento real ordena la población por fitness de mayor a menor y conserva directamente la mitad superior como conjunto de padres. Si el tamaño poblacional es 30, se seleccionan 15 padres en cada generación.

3.4.6. Cruce por Promedio

Se implementó un cruce por promedio. Dados dos padres P_1 y P_2, se genera un único hijo H calculando el promedio de sus genes (pesos) correspondientes:

$$H[w_i] = \frac{P_1[w_i] + P_2[w_i]}{2}$$

Este método genera un nuevo individuo que se ubica en el punto medio del segmento que conecta a los dos padres en el espacio de soluciones. La renormalización de los pesos para que sumen 1 se realiza después del paso de mutación.

3.4.7. Mutación con Ruido Uniforme

Cada gen del individuo muta con una probabilidad $p_m = 0.10$ (10 %). Cuando un gen w_i es seleccionado para mutar, se le suma un valor aleatorio extraído de una distribución uniforme:

$$w_i' = w_i + U(-f_m, f_m), \quad \text{con } f_m = 0.2$$

El valor resultante se recorta para mantenerse en el dominio $[0, 1]$ y, finalmente, el vector completo se renormaliza para asegurar que $\sum w_i' = 1$.

3.4.8. Elitismo

Aunque `config.py` declara un elitismo de 2 individuos, la implementación efectiva preserva solo **un** individuo por generación: el mejor cromosoma encontrado en la generación actual se copia directamente a la nueva población antes de crear el resto de hijos.

3.4.9. Criterio de Parada

El algoritmo genético de la versión actual se detiene únicamente al completar **50 generaciones**. No existe un criterio de *early stopping* por convergencia ni una paciencia configurable aplicada en el código.

Parámetro operativo	Símbolo	Valor
Tamaño de población	N_{pop}	30
Generaciones	T	50
Padres seleccionados	N_{padres}	15
Probabilidad de mutación	p_m	0.10
Factor de mutación	f_m	0.20
Elitismo efectivo	e	1
Longitud del cromosoma	L	5 genes

Cuadro 5: Parámetros realmente usados por el algoritmo genético

Aunque `config.py` declara además `tasa_cruce`, `torneo_size` y `elitismo = 2`, esos campos no gobiernan directamente la lógica de `optimizar_pesos`; la función usa constantes internas equivalentes a la descripción anterior.

3.5. Arquitectura del Sistema

3.5.1. Visión General

El sistema se organiza en cuatro capas de procesamiento que implementan separación de responsabilidades y permiten distinguir claramente la fuente de datos, la lógica espacial, el diagnóstico por zona y la generación de salidas:

1. **Capa de Entrada:** Ingesta y validación de datos de dos fuentes — grafo vial OSM y dataset CSV histórico.

2. **Capa de Procesamiento Espacial:** Construcción de la grilla urbana 30×30 y asignación de nombres de barrios mediante geocodificación inversa.

3. **Capa de IA:** Pipeline operativo por celda: escenario Monte Carlo $\rightarrow$ optimización genética de pesos $\rightarrow$ clasificación difusa $\rightarrow$ ajuste por conectividad y generación de explicación textual.

4. **Capa de Salida:** Generación de mapas HTML, gráficas PNG de históricos y Monte Carlo, y reporte HTML consolidado.

3.5.2. Estructura del Proyecto

```
seguridad/
|-- main.py                     # Punto de entrada principal
|-- config.py                   # Rutas y parámetros declarativos
\-- src/
    |-- algoritmos/
    |   |-- difuso.py           # Clasificación difusa
    |   |-- genetico.py         # Optimización de pesos
    |   |-- reglas.py           # Apriori y PRISM explicativos
    |   |-- montecarlo.py       # Simulación sintética
    |   |-- conectividad.py     # BFS por radio
    |   \-- busqueda.py         # BFS, DFS y A*
    |-- core/
    |   |-- diagnostico.py      # Pipeline central de diagnóstico
    |   |-- mapa.py             # Descarga y grilla OSM
    |   \-- visualizacion.py    # Visualizaciones adicionales
    \-- utils/
        |-- historicos.py       # Gestión del CSV histórico
        \-- reporte.py          # Exportación HTML
```

3.5.3. Flujo de Procesamiento por Zona

Para cada una de las 900 celdas de la grilla 30×30, el sistema ejecuta el siguiente flujo secuencial:

1. Obtener las coordenadas del centro de la celda en la grilla.

2. Determinar la macro-zona comparando la celda con el centro del mapa configurado.

3. Generar un escenario sintético único mediante Monte Carlo a partir de la media, desviación estándar, mínimo y máximo históricos de esa macro-zona.

4. Construir dos métricas auxiliares aleatorias (`densidad_calles` y `densidad_intersecciones`), que hoy se reportan pero no alteran la clasificación difusa.

5. Ejecutar el algoritmo genético para obtener un vector de pesos $\mathbf{w}$ específico para la evaluación actual.

6. Aplicar el clasificador difuso y obtener `nivel` y `score`.

7. Si existe grafo vial cargado, analizar conectividad mediante BFS en radio de 500 m y reajustar el score.

8. Identificar los dos factores con mayor valor y registrar una regla explicativa Apriori o PRISM.
9. Almacenar el diagnóstico y, según el modo de uso, incorporarlo a mapas HTML, consola y reporte.

En la ejecución masiva, la regla explicativa se calcula después del ajuste por conectividad. En cambio, el diagnóstico detallado mostrado por consola la imprime antes de recalcular el nivel final con conectividad. Por ello, la regla textual debe interpretarse como un apoyo descriptivo del proceso y no como una justificación garantizada del nivel definitivo en todos los modos de uso.

3.5.4. Configuración Centralizada

El módulo `config.py` concentra rutas, colores y varios parámetros declarativos del sistema; sin embargo, la implementación operativa combina esos valores con constantes internas definidas dentro de los módulos algorítmicos. La Tabla 6 resume los valores realmente observables en el código.

Componente	Parámetro	Valor operativo
Grid	Tamaño	30×30 (900 zonas)
Conectividad	Radio BFS	500 metros
Algoritmo genético	Población / generaciones	30 individuos / 50 generaciones
Algoritmo genético	Selección / elitismo	mitad superior como padres / elitismo efectivo de 1 individuo
Algoritmo genético	Mutación	probabilidad 0.10 y factor uniforme 0.20
Clasificador difuso	Tipos de membresía	triangular para robos, vandalismo y accidentes; sigmoidal para microtráfico y llamadas al 911
Clasificación final	Umbrales por score	Bajo < 33, Medio < 66, Alto en caso contrario
Reglas	Apriori / PRISM	soporte 0.10, confianza 0.70 / confianza 0.50
Monte Carlo	Uso operativo	1 escenario por celda durante el diagnóstico; 1000 simulaciones por macro-zona para visualización

Cuadro 6: Parámetros operativos del sistema

3.6. Metodología de Integración

3.6.1. Simulación Monte Carlo

La simulación Monte Carlo implementada en `src/algoritmos/montecarlo.py` se basa en muestreo normal acotado por los valores históricos mínimos y máximos de cada macro-zona [3]. Para una variable j, el valor simulado se genera como:

$$X_j = \text{clip}\big(\mathcal{N}(\mu_j^{zona}, \sigma_j^{zona}),\ min_j^{zona},\ max_j^{zona}\big) \tag{10}$$

En la ejecución operativa del diagnóstico por celda se utiliza un **solo escenario** generado por

macro-zona y celda. El valor $N = 1000$ aparece en el proyecto únicamente para generar visualizaciones comparativas y reportes Monte Carlo.

Esto permite:

- producir escenarios sintéticos consistentes con los rangos históricos;
- comparar visualmente mínimos, medias y máximos históricos frente a simulaciones mediante barras agrupadas;
- generar resúmenes comparativos por macro-zona en consola y, en el reporte HTML actual, un resumen centrado en la variable robos.

3.6.2. Análisis de Conectividad BFS

Para cada celda se ejecuta una búsqueda en amplitud (BFS) desde el nodo OSM más cercano al centroide de la zona. El algoritmo recorre vecinos mientras su distancia Haversine al centro de la zona no exceda los 500 m. El número de nodos alcanzables clasifica la conectividad vial como:

- **Baja:** menos de 10 nodos alcanzables, con `factor_riesgo = 0.8`.
- **Media:** entre 10 y 29 nodos, con `factor_riesgo = 0.5`.
- **Alta:** 30 nodos o más, con `factor_riesgo = 0.2`.

La conectividad se incorpora al score final mediante la fórmula:

$$score_{ajustado} = score_{difuso} \cdot \frac{1 + factor_riesgo}{2} \tag{11}$$

Por tanto, una zona con baja conectividad conserva una fracción mayor del score difuso que una zona altamente conectada. Tras este ajuste, el nivel final se vuelve a discretizar con los umbrales 33 y 66.

3.6.3. Procesamiento Geográfico

El sistema implementa las siguientes operaciones sobre el grafo OSM:

- **Construcción de grilla:** Particionamiento del área descargada desde OSM en una matriz regular de $30 \times 30 = 900$ celdas.
- **Geocodificación inversa:** Para puntos de referencia de la grilla, el sistema consulta la API Nominatim/OSM y guarda el resultado en caché local. Se usa un muestreo cada 6 celdas y luego se propaga el nombre más cercano al resto, reduciendo las consultas a aproximadamente 25 por ciudad.
- **Asignación de macro-zonas:** La macro-zona se determina comparando cada coordenada con el centro del mapa: noreste $\rightarrow$ Norte, noroeste $\rightarrow$ Oeste, sureste $\rightarrow$ Este y suroeste $\rightarrow$ Sur.

3.7. Análisis de Resultados

3.7.1. Distribución de Riesgo en la Ejecución

Tras la ejecución del sistema sobre la grilla completa o sobre una zona circular, el programa resume el número de celdas clasificadas como *Bajo*, *Medio* y *Alto*. Debido a que el pipeline incorpora generación aleatoria de escenarios Monte Carlo y un algoritmo genético sin semilla fija, estas proporciones pueden variar entre ejecuciones. En consecuencia, el código no garantiza porcentajes estáticos por macro-zona; lo que sí produce de forma consistente es un conteo agregado de niveles y un detalle por zona analizada.

3.7.2. Impacto del Algoritmo Genético

El algoritmo genético optimiza los pesos de ponderación de las variables a lo largo de 50 generaciones y reporta periódicamente el mejor fitness alcanzado. En el código actual no existe una comparación automática contra una línea base fija ni una medición formal de mejora porcentual respecto a pesos manuales; por ello, el impacto del AG debe interpretarse como una calibración heurística de pesos y no como una mejora cuantificada automáticamente por el sistema.

3.7.3. Coherencia con Datos Históricos

La coherencia con los datos históricos se observa en dos niveles implementados en el código:

- comparación visual entre medias, mínimos y máximos históricos y simulados en las gráficas Monte Carlo;
- resumen numérico de medias históricas y simuladas para la variable de robos dentro del reporte HTML actual y de la salida por consola.

No se implementan actualmente métricas automáticas como correlación de Spearman, intervalos de credibilidad bayesianos ni cobertura sistemática de las reglas explicativas generadas.

3.7.4. Visualizaciones Generadas

El sistema produce los siguientes artefactos digitales:

- **Mapa HTML interactivo (Folium):** Cada celda se representa con un `CircleMarker` coloreado según su nivel de riesgo e incluye un *popup* con factores principales y regla explicativa.
- **Gráficas PNG de series históricas del CSV:** El programa exporta curvas suaves de distribución para los 12 meses del archivo histórico, mostrando media, rango mínimo–máximo y zona $\pm 1\sigma$ para las cuatro macro-zonas.
- **Gráficas PNG de Monte Carlo:** Barras agrupadas por variable y macro-zona comparando mínimos, medias y máximos históricos frente a los valores simulados (1000 simulaciones para visualización).
- **Reporte HTML consolidado:** Resume conteos por nivel, tablas históricas por macro-zona, una comparación Monte Carlo resumida por zona sobre la variable robos y el detalle de zonas analizadas.

- **Salida por consola:** Presenta trazabilidad paso a paso del diagnóstico detallado cuando se analiza una zona o un conjunto circular.

3.8. Limitaciones

1. **Calidad y obsolescencia de los datos históricos:** El sistema depende críticamente de la frescura del archivo CSV de históricos. Si los datos no se actualizan periódicamente (recomendado: mensual), el diagnóstico puede reflejar dinámicas delictivas desactualizadas y sesgar las predicciones.

2. **Ausencia de validación de campo:** Los diagnósticos generados por el sistema no han sido contrastados sistemáticamente con datos de intervención policial real ni con la percepción de los habitantes de cada barrio. Sin esta validación, el sistema constituye un insumo de apoyo a la decisión, no una verdad operacional.

3. **Simplificación de la realidad urbana:** La discretización en una grilla regular 30×30 ignora la heterogeneidad interna de las celdas, que pueden contener microentornos con dinámicas muy distintas (zonas comerciales vs. residenciales dentro de la misma celda).

4. **Dependencia de conectividad a OpenStreetMap:** La descarga del grafo vial requiere acceso a internet al momento de ejecución. En redes inestables, esto puede interrumpir el procesamiento o generar grafos incompletos.

5. **Dependencia de entorno gráfico para el modo detallado:** El modo 2 del sistema requiere `tkinter/Tk` para abrir la interfaz interactiva de selección de zona. Si este soporte no está disponible, el programa redirige automáticamente al modo 1 con backend gráfico no interactivo.

6. **Reglas explicativas no condicionadas por la celda actual:** Apriori y PRISM se ejecutan sobre un dataset sintético interno generado en el propio módulo de reglas, por lo que la explicación textual no surge directamente del historial particular de la celda analizada.

7. **Desalineación puntual entre regla y nivel final en modo detallado:** En el diagnóstico paso a paso, la regla explicativa se imprime antes del ajuste final por conectividad. Si dicho ajuste modifica el nivel, el texto mostrado puede no coincidir exactamente con la clasificación definitiva.

8. **Variables auxiliares no integradas al score difuso:** El pipeline genera métricas de densidad vial e intersecciones, pero estas no participan en el cálculo del clasificador difuso actual.

9. **Síntesis de características desde distribución normal:** El muestreo Monte Carlo asume que los incidentes siguen una distribución normal por macro-zona, lo cual puede ser inexacto para variables con distribuciones sesgadas o bimodales (por ejemplo, robos con alta variabilidad estacional).

10. **Ausencia de semilla fija:** La simulación Monte Carlo y varios pasos del algoritmo genético son estocásticos; por ello, dos ejecuciones sucesivas pueden producir resultados distintos.

3.9. Conclusiones

El desarrollo realizado permitió consolidar un sistema integral de diagnóstico de riesgo urbano para la ciudad de Ambato, articulando de manera funcional varias técnicas de Inteligencia Artificial dentro

de un mismo flujo de procesamiento. La integración de simulación Monte Carlo, lógica difusa, algoritmo genético, análisis de conectividad mediante BFS y reglas explicativas Apriori/PRISM evidenció que es posible construir un prototipo analítico capaz de transformar datos históricos y geográficos en resultados interpretables, útiles para la caracterización preliminar de zonas con distinto nivel de riesgo.

Desde el punto de vista metodológico, la investigación confirma que la lógica difusa constituye el núcleo más adecuado para representar la incertidumbre inherente al fenómeno estudiado, al permitir la transición entre categorías de riesgo sin recurrir a umbrales estrictamente binarios. A ello se suma el aporte del algoritmo genético, que automatiza la calibración de pesos de las variables consideradas, y el componente basado en reglas, que fortalece la interpretabilidad del diagnóstico al ofrecer explicaciones legibles para los niveles medio y alto. En conjunto, estos elementos no solo mejoran la consistencia interna del sistema, sino que también incrementan su valor como herramienta académica y demostrativa.

En términos de resultados, el proyecto demuestra que un enfoque híbrido puede generar salidas técnicamente consistentes y comunicativamente útiles, tales como mapas HTML, gráficas comparativas, trazas detalladas por consola y reportes consolidados. No obstante, también se constata que la precisión operativa del sistema depende de la calidad y actualización de los datos históricos, así como de futuras etapas de validación empírica. Por ello, el trabajo debe entenderse como una base sólida y metodológicamente coherente para posteriores procesos de refinamiento, validación institucional y eventual aplicación en escenarios reales de apoyo a la toma de decisiones en seguridad ciudadana.

3.10. Recomendaciones

Como primera recomendación, se considera indispensable fortalecer la base empírica del sistema mediante la actualización periódica del dataset histórico y su contraste con registros oficiales verificados. La utilidad analítica del diagnóstico depende directamente de la vigencia, granularidad y confiabilidad de la información empleada; por ello, una futura implementación debería vincularse con fuentes institucionales de mayor resolución espacial y temporal, de modo que el modelo pueda reflejar con mayor fidelidad la dinámica real de la seguridad urbana.

En segundo término, se recomienda ampliar el alcance metodológico del sistema incorporando nuevas variables contextuales y extendiendo el proceso de optimización. Factores como densidad comercial, iluminación pública, nivel socioeconómico, presencia de equipamientos de seguridad o patrones temporales de incidencia podrían enriquecer significativamente la capacidad explicativa del modelo. De igual manera, resultaría pertinente que el algoritmo genético optimice no solo los pesos de entrada, sino también parámetros de las funciones de membresía, con el propósito de lograr una representación más fina y adaptativa del riesgo.

Finalmente, se sugiere orientar el trabajo hacia una fase de validación aplicada e interoperabilidad institucional. La comparación sistemática entre los diagnósticos generados por el sistema y evidencias de campo permitiría estimar su precisión real, identificar sesgos y establecer criterios de mejora continua. Bajo esta perspectiva, la arquitectura modular desarrollada ofrece una base adecuada para escalar el prototipo hacia otros contextos urbanos del país, siempre que dicho proceso se acompañe de validación técnica, coordinación interinstitucional y criterios rigurosos de actualización y supervisión de datos.

3.11. Habilidades blandas empleadas en la práctica

☐ Liderazgo
■ Trabajo en equipo
☐ Comunicación asertiva

☐ La empatía
☐ Pensamiento crítico
☐ Flexibilidad
☐ La resolución de conflictos
☐ Adaptabilidad
■ Responsabilidad

4 APÉNDICE: DIAGRAMAS Y ESQUEMAS

La siguiente figura ilustra la arquitectura general del sistema, mostrando cómo fluyen los datos desde las fuentes de entrada (OpenStreetMap y datos históricos) a través del motor de diagnóstico, donde se aplican los algoritmos de IA, hasta la generación de salidas (mapas y gráficas):

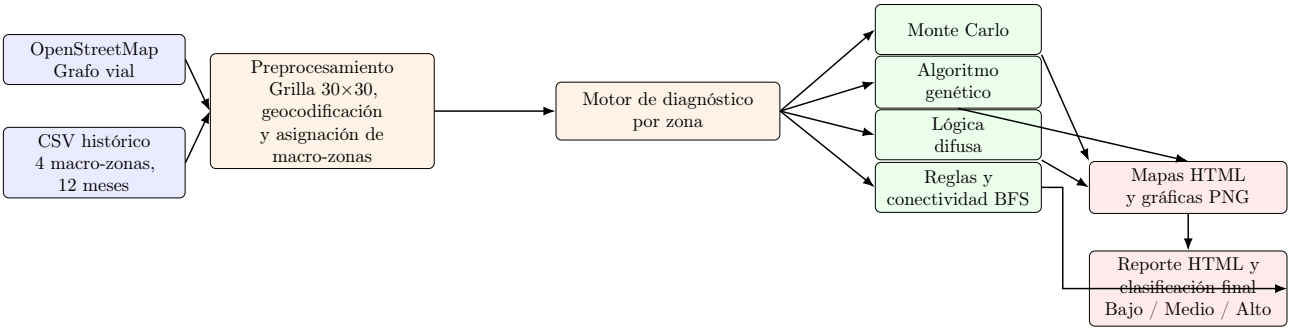

Figura 1: Arquitectura del Sistema

Fuente: Elaboración propia con base en la arquitectura del sistema implementado.

El siguiente diagrama detalla el flujo secuencial de procesamiento para cada una de las 900 zonas de análisis:

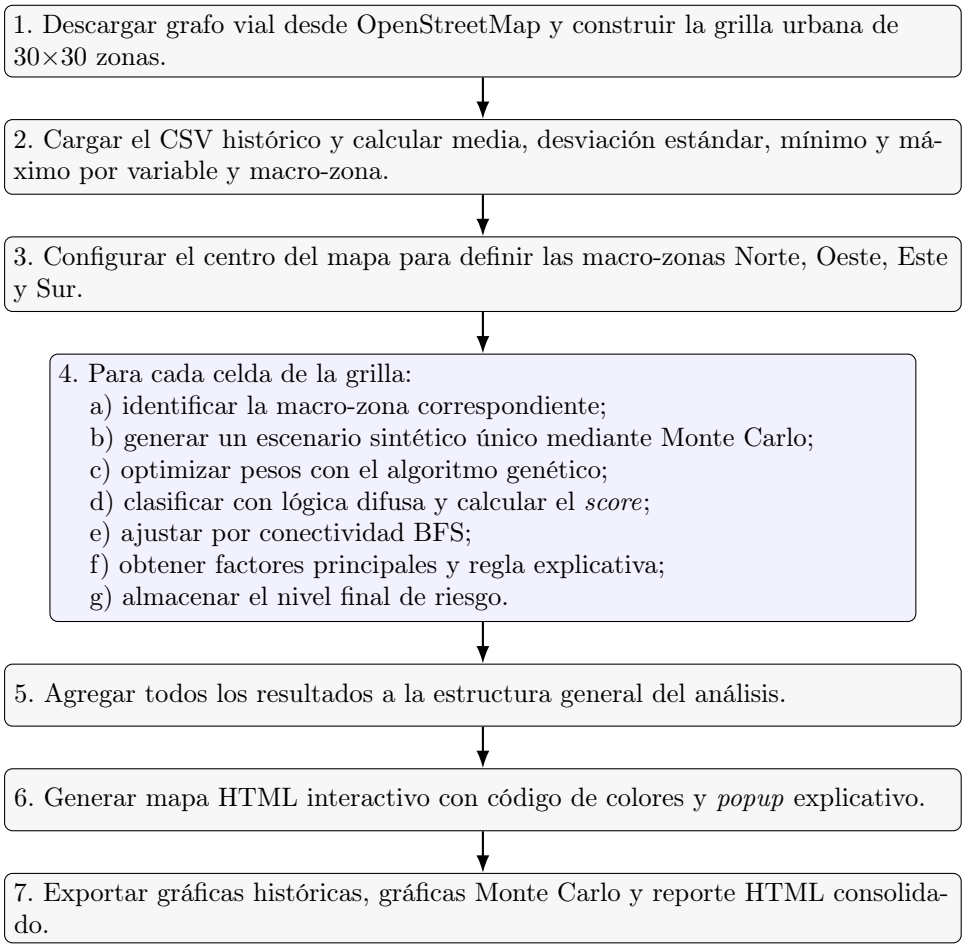

Figura 2: Flujo de Procesamiento por Zona

Fuente: Elaboración propia con base en el flujo operacional del sistema desarrollado.

4.1. Referencias bibliográficas

[1] L. A. Zadeh, *Fuzzy Sets*, *Information and Control*, Vol. 8, No. 3, pp. 338–353, 1965.

[2] J. H. Holland, *Adaptation in Natural and Artificial Systems*, University of Michigan Press, 1975.

[3] G. E. P. Box and M. E. Muller, *A Note on the Generation of Random Normal Deviates*, *The Annals of Mathematical Statistics*, Vol. 29, No. 2, pp. 610–611, 1958.

[4] R. Agrawal and R. Srikant, *Fast Algorithms for Mining Association Rules*, *Proceedings of the 20th International Conference on Very Large Data Bases*, pp. 487–499, 1994.

[5] J. R. Quinlan, *Learning Logical Definitions from Relations*, *Machine Learning*, Vol. 5, No. 3, pp. 239–266, 1990.

[6] E. H. Mamdani and S. Assilian, *An Experiment in Linguistic Synthesis with a Fuzzy Logic Controller*, *International Journal of Man-Machine Studies*, Vol. 7, No. 1, pp. 1–13, 1975.

[7] G. Boeing, *OSMnx: New Methods for Acquiring, Constructing, Analyzing, and Visualizing Complex Street Networks*, *Computers, Environment and Urban Systems*, Vol. 65, pp. 126–139, 2017.